

BCSE204L Design and Analysis of Algorithms

Module-2

Topic-2: Dynamic Programming-problems

Edit distance

PROBLEM:

- **Spell checker** encounters a possible misspelling, it looks in its dictionary for other words that are close by. What is the appropriate notion of closeness in this case?
 - Measure of the distance between two strings is the extent to which they can be aligned, or matched up
 - An alignment is simply a way of writing the strings one above the other
- Two possible alignments of SNOWY and SUNNY:

S	—	N	O	W	Y
S	U	N	N	—	Y

Cost: 3

—	S	N	O	W	—	Y
S	U	N	—	—	N	Y

Cost: 5

- “—” indicates a “gap”; any number of these can be placed in either string
- **Cost** of an alignment is the number of columns in which the letters differ
- **Edit distance** between two strings is the cost of their best possible alignment
- Edit distance means the minimum number of edits—**insertions, deletions, and substitutions** of characters—needed to transform the first string into the second
- For the first the alignment, edit distance 3 corresponds to three edits: insert U, substitute O → N, and delete W.

A dynamic programming solution

- For DP problem, the most crucial question is, What are the subproblems?
 - ❖ **Goal** is to find the edit distance between two strings $x[1 \cdots m]$ and $y[1 \cdots n]$
- Subproblem will be finding the edit distance between some prefix of the first string, $x[1 \cdots i]$, and some prefix of the second, $y[1 \cdots j]$

E	X	P	O	N	E	N
---	---	---	---	---	---	---

 T I A L

P	O	L	Y	N
---	---	---	---	---

 O M I A L

- If the subproblem is $E(i, j)$, our final objective, then, is to compute $E(m, n)$
- For computing the best alignment between $x[1 \cdots i]$ and $y[1 \cdots j]$ rightmost column can only be one of three things:

$x[i]$
— or — or $x[i]$
 $y[j]$ $y[j]$

$$\begin{array}{c} x[i] \\ - \end{array} \quad \text{or} \quad \begin{array}{c} - \\ y[j] \end{array} \quad \text{or} \quad \begin{array}{c} x[i] \\ y[j] \end{array}$$

- The first case incurs a cost of 1 for this particular column, and it remains to align $x[1 \dots i - 1]$ with $y[1 \dots j]$ -----> subproblem **$E(i-1, j)$**
 - The second case, also with cost 1, still need to align $x[1 \dots i]$ with $y[1 \dots j - 1]$. -----> subproblem **$E(i, j - 1)$**
 - The final case, which either costs 1 (if $x[i] == y[j]$) or 0 (if $x[i] != y[j]$), ----->subproblem **$E(i - 1, j - 1)$**
- Thus, we have expressed $E(i, j)$ in terms of **three smaller subproblems** $E(i - 1, j)$, $E(i, j - 1)$, $E(i - 1, j - 1)$
- As we have no idea which of them is the right one, so we need to try them all and pick the best:
- $$E(i, j) = \min\{1 + E(i - 1, j), 1 + E(i, j - 1), \text{diff}(i, j) + E(i - 1, j - 1)\}$$

subproblem $E(4, 3)$ corresponds to the prefixes EXPO and POL. The rightmost column of their best alignment must be one of the following:

$$\begin{array}{c} \text{O} \\ - \end{array} \quad \text{or} \quad \begin{array}{c} - \\ \text{L} \end{array} \quad \text{or} \quad \begin{array}{c} \text{O} \\ \text{L} \end{array}$$

Thus, $E(4, 3) = \min\{1 + E(3, 3), 1 + E(4, 2), 1 + E(3, 2)\}$.

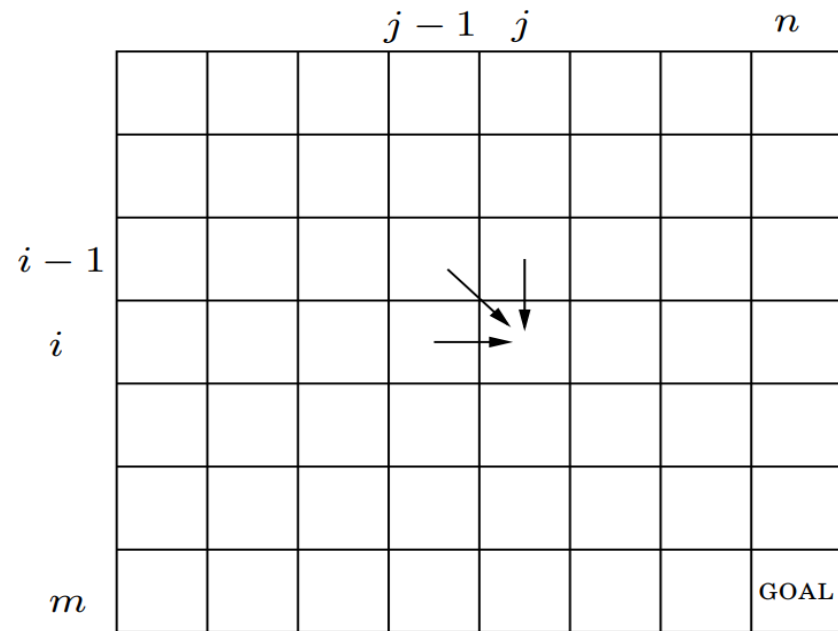
- The answers to all the subproblems $E(i, j)$ form a two-dimensional table
- In what order should these subproblems be solved?
 - If we follow iterative procedure, $E(i - 1, j)$, $E(i, j - 1)$, and $E(i - 1, j - 1)$ are handled before $E(i, j)$.
 - For instance, we could fill in the table one row at a time, from top row to bottom row, and moving left to right across each row. Or alternatively, we could fill it in column by column.

```

for  $i = 0, 1, 2, \dots, m$ :
     $E(i, 0) = i$ 
for  $j = 1, 2, \dots, n$ :
     $E(0, j) = j$ 
for  $i = 1, 2, \dots, m$ :
    for  $j = 1, 2, \dots, n$ :
         $E(i, j) = \min\{E(i - 1, j) + 1, E(i, j - 1) + 1, E(i - 1, j - 1) + \text{diff}(i, j)\}$ 
return  $E(m, n)$ 

```

(a)



(b)

		P	O	L	Y	N	O	M	I	A	L
E	0	1	2	3	4	5	6	7	8	9	10
X	1	1	2	3	4	5	6	7	8	9	10
P	2	2	2	3	4	5	6	7	8	9	10
O	3	2	3	3	4	5	6	7	8	9	10
N	4	3	2	3	4	5	5	6	7	8	9
E	5	4	3	3	4	4	5	6	7	8	9
N	6	5	4	4	4	5	5	6	7	8	9
T	7	6	5	5	5	4	5	6	7	8	9
I	8	7	6	6	6	5	5	6	7	8	9
A	9	8	7	7	7	6	6	6	6	7	8
L	10	9	8	8	8	7	7	7	7	6	7
	11	10	9	8	9	8	8	8	8	7	6

the edit distance turns out to be 6:

E	X	P	O	N	E	N	—	T	I	A	L
—	—	P	O	L	Y	N	O	M	I	A	L

Longest Common Subsequence (LCS)

- The longest common subsequence (LCS) problem is the problem of finding the longest subsequence common to all sequences in a set of sequences
- Unlike substrings, subsequences are **NOT** required to occupy consecutive positions within the original sequences

LCS Example

- Seq1: ($ABCD$), Seq2: ($ACBAD$)
- Length-2 common subsequences:
(AB), (AC), (AD), (BD), and (CD), totaling to 5
- Length-3 common subsequences: (ABD) and (ACD), totaling to 2
- No common subsequences longer than 3
- So (ABD) and (ACD) are their longest common subsequences and length of LCS = 3

LCS DP- The idea

- Let the input sequences be $X[0..m - 1]$ and $Y[0..n - 1]$ of lengths m and n respectively.
- Let $L(X[0..m - 1], Y[0..n - 1])$ be the length of LCS of the two sequences X and Y .
- The recursive definition of $L(X[0..m - 1], Y[0..n - 1])$ can be framed based on whether the last characters in the previous call match or not

LSP DP- The idea

- If last characters of both sequences match (or $X[m - 1] == Y[n - 1]$) then
$$\begin{aligned} &L(X[0..m - 1], Y[0..n - 1]) \\ &= 1 + L(X[0..m - 2], Y[0..n - 2]) \end{aligned}$$
- If last characters of both sequences do not match (or $X[m - 1] \neq Y[n - 1]$) then
$$\begin{aligned} &L(X[0..m - 1], Y[0..n - 1]) = \\ &\text{MAX}\{L(X[0..m - 2], Y[0..n - 1]), L(X[0..m - 1], Y[0..n - 2])\} \end{aligned}$$

LCS DP- An Example

- Consider the input strings “*AGGTAB*” and “*GXTXAYB*”.
Last characters match for the strings

- So length of LCS can be written as:

$$L(AGGTAB, GXTXAYB) = 1 + L(AGGTA, GXTXAY)$$

- Consider the input strings “*ABCDGH*” and “*AEDFHR*”.
Last characters do not match for the strings

- So length of LCS can be written as: $L(ABCDGH, AEDFHR) = \text{MAX} \{L(ABCDG, AEDFHR), L(ABCDGH, AEDFH)\}$

LCS DP Pseudocode

```
X and Y be two given sequences
Initialize a table LCS of dimension X.length * Y.length
X.label = X
Y.label = Y
LCS[0][] = 0
LCS[][0] = 0
Start from LCS[1][1]
Compare X[i] and Y[j]
    If X[i] = Y[j]
        LCS[i][j] = 1 + LCS[i-1, j-1]
    Else
        LCS[i][j] = max(LCS[i-1][j], LCS[i][j-1])
```

		0	1	2	3	4 = n
		B D C B				
0		0	0	0	0	0
1	B	0	1	1	1	1
2	A	0	1	1	1	1
3	C	0	1	1	2	2
4	D	0	1	2	2	2
$m=5$	B	0	1	2	2	3

LCS Length Table

$X = \text{BACDB}$

$Y = \text{BDCB}$

LCS = BCB

		0	1	2	3	4 = n
		B D C B				
0		0	0	0	0	0
1	B	0	1	1	1	1
2	A	0	1	1	1	1
3	C	0	1	1	2	2
4	D	0	1	2	2	2
$m=5$	B	0	1	2	2	3

← start here

with back pointers included

- Find LCS of **ABCBDA**B and BDCABA

		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

Analysis

- Time complexity
- If m and n are the lengths of the sequences
- Then filling an $m \times n$ table
- Therefore, $O(mn)$